REVISION NOTES • TOPIC 8

const, readonly, static and Object Lifetime

Simple explanations • examples • Fundraising Platform • diagrams • interview preparation

Topic Coverage

This topic explains:

- fields and local variables;
- instance fields;
- static fields;
- `const`;
- compile-time constants;
- constant expressions;
- allowed `const` types;
- `readonly`;
- instance `readonly`;
- constructor assignment rules;
- `static readonly`;
- `const` vs `readonly`;
- `const` vs `static readonly`;
- binary/versioning differences;
- `static`;
- static fields;
- static properties;
- static methods;
- static classes;
- static constructors;
- instance constructors vs static constructors;
- initialization order;
- object instance lifetime;
- static member lifetime;
- GC roots;
- how static references can keep objects alive;
- shared mutable static state;
- thread-safety problems;
- appropriate use in ASP.NET Core;
- when to use and when not to use each feature;
- Fundraising Platform examples;

- interview questions;
- practical exercises.

The most important high-level distinction is:

```
C#
const
    →
Compile-Time Constant

readonly
    →
Assigned During Initialization
Then Cannot Be Reassigned

static
    →
Belongs to Type
Shared Across Instances
```

These concepts can also be combined:

```
C#
static readonly
    →
One Shared Field
Initialized at Runtime
Then Cannot Be Reassigned
```

✦ First Understand: Field vs Local Variable

Before understanding `const`, `readonly`, and `static`, you must understand where a variable belongs.

Consider:

```
C#
public sealed class Gift
{
    private decimal _totalAmount;

    public void Process()
    {
        decimal processingFee = 10m;
    }
}
```

Here:

EXAMPLE / FLOW

```
_totalAmount
    →
Field

processingFee
    →
Local Variable
```

✦ What Is a Field?

Definition

A **field** is a variable declared directly inside a class or struct.

Example:

C#

```
public sealed class Gift
{
    private decimal _totalAmount;
}
```

`_totalAmount` belongs to the `Gift` type.

Fields represent part of the state of a type or object.

Microsoft describes fields as variables that contain part of a type's state.

✦ What Is a Local Variable?

A local variable exists inside:

EXAMPLE / FLOW

```
Method

Constructor

Loop

Block

Local Function
```

Example:

C#

```
public void ProcessGift()
{
    decimal fee = 25m;
}
```

`fee` exists only inside that method scope.

✦ Instance Field

An **instance field** belongs to each individual object.

Example:

```C#
public sealed class Gift
{
    private decimal _totalAmount;
}
```

Create:

```C#
var gift1 = new Gift();
var gift2 = new Gift();
```

Conceptually:

EXAMPLE / FLOW

```
gift1
    →
Gift Object A
    →
_totalAmount

gift2
    →
Gift Object B
    →
_totalAmount
```

Each object has its own field value.

The C# specification states that each class instance has its own complete set of instance fields.

✦ Static Field

A static field belongs to the **type itself** rather than an individual object.

Example:

```C#
public sealed class Gift
{
    public static int CreatedCount;
}
```

Conceptually:

EXAMPLE / FLOW

```
Gift Type
    →
CreatedCount

Gift Object A
Gift Object B
Gift Object C
    →
all share the same CreatedCount
```

For a non-generic class there is one set of static fields for the type, regardless of how many instances exist. Closed generic types have their own static state per closed constructed type.

✦ Instance vs Static Field

Consider:

C#

```
public sealed class Gift
{
    public decimal TotalAmount;

    public static int GiftCount;
}
```

If we create:

C#

```
var gift1 = new Gift
{
    TotalAmount = 100m
};

var gift2 = new Gift
{
    TotalAmount = 500m
};
```

Conceptually:

EXAMPLE / FLOW

```
gift1.TotalAmount
    →
100

gift2.TotalAmount
    →
500
```

But:

```C#
Gift.GiftCount = 2;
```

is shared:

EXAMPLE / FLOW

```
Gift Type
    →
GiftCount = 2
```

There is not a separate `GiftCount` for each gift.

✦ What Is `const`?

Definition

`const` declares a **compile-time constant**.

A constant:

- must receive its value when declared;
- cannot later be changed;
- must be evaluable at compile time;
- belongs conceptually to the type when declared as a field;
- does not use the `static` modifier because constants already behave as type-level members.

Microsoft defines constants as immutable values known at compile time and requires them to be initialized at declaration.

✦ Basic `const` Syntax

```C#
const int MaxRetryCount = 3;
```

Example:

```C#
const decimal MinimumGiftAmount = 1m;
```

Example:

```C#
const string DefaultCurrencyCode = "USD";
```

✦ const Field

Inside a type:

```C#
public static class GiftRules
{
    public const decimal MinimumGiftAmount = 1m;
}
```

Usage:

```C#
decimal minimum =
    GiftRules.MinimumGiftAmount;
```

Notice:

EXAMPLE / FLOW

```
No object instance required.
```

A constant field behaves like a type-level member even though you do not write `static`. Microsoft explicitly notes that `static const` is not allowed because a constant field is already static-like in how it belongs to the type.

✦ This Is Invalid

```C#
public static const int MaxRetries = 3;
```

Why?

Because:

EXAMPLE / FLOW

```
const
already has type-level behavior.
```

Correct:

```C#
public const int MaxRetries = 3;
```

✦ Local Constants

`const` can also be used inside a method.

Example:

```C#
public void ProcessGift()
{
    const int MaxRetries = 3;

    // use MaxRetries
}
```

This is a local constant.

It exists only within that scope.

✦ const Must Be Initialized Immediately

This is valid:

```C#
const int MaxRetries = 3;
```

This is invalid:

```C#
const int MaxRetries;

// MaxRetries = 3;
```

A `const` declaration must contain its initializer.

✦ const Cannot Be Changed

```C#
const int MaxRetries = 3;

// Invalid:
// MaxRetries = 5;
```

After compilation:

EXAMPLE / FLOW

```
MaxRetries
=
3
```

is fixed for that compilation.

✦ What Does Compile-Time Constant Mean?

A compile-time constant is something the compiler can fully determine while compiling the application.

Example:

```C#
const int First = 10;
const int Second = 20;

const int Total =
    First + Second;
```

The compiler can calculate:

```EXAMPLE / FLOW
10 + 20
=
30
```

during compilation.

✦ Constant Expression

A constant expression can be fully evaluated by the compiler.

Example:

```C#
const int BaseCount = 5;

const int MaximumCount =
    BaseCount * 10;
```

Result:

```EXAMPLE / FLOW
MaximumCount = 50
```

before the application starts running.

The C# specification defines constant expressions as expressions fully evaluable at compile time.

✦ This Is Not a Compile-Time Constant

```C#
const DateTime StartedAt =
    DateTime.UtcNow;
```

Invalid.

Why?

```EXAMPLE / FLOW
DateTime.UtcNow
```

is determined at runtime.

The compiler cannot know the current time during compilation.

✦ Use readonly Instead

```C#
public static readonly DateTime StartedAtUtc =
    DateTime.UtcNow;
```

Now the value is calculated:

EXAMPLE / FLOW

```
Runtime
    →
DateTime.UtcNow
    →
assign once
```

✦ Which Types Can Be const?

Constants can use appropriate built-in/simple value types, enumeration types, strings, and null reference constants according to the language's constant-expression rules. User-defined structs/classes/arrays cannot generally be created as non-null `const` values.

Common practical constant types:

EXAMPLE / FLOW

```
int
long
byte
short

float
double
decimal

bool
char

string

enum
```

✦ Examples

```C#
const int MaxRetries = 3;

const decimal MinimumAmount = 1m;

const bool LoggingEnabled = true;

const char Separator = ',';

const string Currency = "USD";
```

✦ Enum Constants

Example:

```C#
public enum GiftType
{
    Cash = 1,
    Check = 2,
    Electronic = 3
}
```

Enums naturally define named constant values.

✦ Reference Type Constants

This is valid:

```C#
const string ProductName =
    "Fundraising Platform";
```

String is special because string literals can participate in constant expressions.

For other reference types, the only normal constant value available is `null`.

✦ This Is Invalid

```C#
const Gift DefaultGift =
    new Gift();
```

Why?

EXAMPLE / FLOW

```
new Gift()
```

happens at runtime.

It is not a compile-time constant expression.

Use:

```C#
static readonly
```

instead if a shared initialized object is needed.

✦ Interpolated Constant Strings

Modern C# can create a constant interpolated string when all inserted expressions are also constant strings.

Example:

```C#
const string Application =
    "Fundraising";

const string Platform =
    ".NET";

const string DisplayName =
    $"{Application} - {Platform}";
```

This is permitted when all parts satisfy the constant-string requirements.

✦ When Should You Use const?

Use `const` when the value is:

```EXAMPLE / FLOW
Known at compile time

Truly fixed

Unlikely to change independently of compiled callers

Naturally part of source-code rules
```

Examples:

```C#
public const int MonthsInYear = 12;
```

```C#
public const string UsdCurrencyCode = "USD";
```

depending on design.

✦ Fundraising Example

```C#
public static class GiftValidationRules
{
    public const decimal MinimumGiftAmount =
        0.01m;
}
```

Usage:

```C#
if (giftAmount <
    GiftValidationRules.MinimumGiftAmount)
{
    throw new ArgumentOutOfRangeException(
        nameof(giftAmount));
}
```

✦ What Should Not Be const?

Do not use `const` for values likely to change operationally.

Examples:

```EXAMPLE / FLOW
API URL

Maximum API Retry Count from configuration

Database Connection String

Campaign Goal

Processing Fee Percentage

Feature Flag

Application Version Display

Company Name if externally configurable
```

Why?

Those are usually:

```EXAMPLE / FLOW
configuration
or
runtime state
```

rather than compile-time invariants.

Microsoft specifically warns against using constants for information expected to change over time because const values can be propagated into compiled callers.

✦ Important const Versioning Problem

This is a professional-level point.

Suppose Library A contains:

```C#
public const int MaxRetries = 3;
```

Application B references it:

```C#
int retries =
    LibraryA.MaxRetries;
```

When Application B is compiled, the compiler can embed:

```EXAMPLE / FLOW
3
```

directly into Application B.

Now later Library A changes:

```C#
public const int MaxRetries = 5;
```

If only Library A is replaced but Application B is **not recompiled**, Application B may still use:

```EXAMPLE / FLOW
3
```

because that value was compiled into the caller.

The C# specification explicitly distinguishes this versioning behavior from `static readonly`.

✦ const Versioning Flow

Initial:

```EXAMPLE / FLOW
Library
MaxRetries = 3
    →
Compile Consumer
    →
3 embedded into consumer
```

Later:

EXAMPLE / FLOW

```
Library changed to 5
    →
Consumer not rebuilt
    →
Consumer may still use 3
```

✦ Why static readonly Can Be Better for Public Library Values

Suppose:

C#

```
public static readonly int MaxRetries =
    3;
```

The consumer reads the field at runtime.

Change library:

C#

```
public static readonly int MaxRetries =
    5;
```

A separately compiled consumer can observe the new runtime field value without the same compile-time constant inlining behavior.

This is one reason public library constants deserve careful consideration.

✦ What Is readonly?

Definition

A `readonly` field can be assigned only during controlled initialization.

For an instance field, direct assignment is allowed:

EXAMPLE / FLOW

```
At field declaration

or

Inside an instance constructor
of the containing type
```

After object construction, the field cannot be reassigned.

The C# specification defines these assignment restrictions directly.

✦ Basic readonly Field

```C#
public sealed class Gift
{
    private readonly Guid _giftTrackingId =
        Guid.NewGuid();
}
```

The value is created at runtime.

After initialization:

EXAMPLE / FLOW

```
_giftTrackingId
```

cannot be reassigned by normal code.

✦ readonly in Constructor

```C#
public sealed class Gift
{
    private readonly long _tenantId;

    public Gift(
        long tenantId)
    {
        _tenantId =
            tenantId;
    }
}
```

This is valid.

✦ Different Objects Can Have Different readonly Values

This is very important.

```C#
var gift1 =
    new Gift(
        tenantId: 1);

var gift2 =
    new Gift(
        tenantId: 2);
```

Conceptually:

EXAMPLE / FLOW

```
gift1
    →
_tenantId = 1

gift2
    →
_tenantId = 2
```

`readonly` does not mean:

EXAMPLE / FLOW

```
same value for all objects
```

It means:

EXAMPLE / FLOW

```
once this field is initialized
for this object,
its field reference/value cannot be reassigned.
```

✦ readonly Assignment Flow

EXAMPLE / FLOW

```
Object Construction
    →
Field initializer
or
Constructor
    →
Assign readonly field
    →
Construction finishes
    →
Field cannot be reassigned normally
```

✦ This Is Invalid

```C#
public sealed class Gift
{
    private readonly long _tenantId;

    public Gift(
        long tenantId)
    {
        _tenantId =
            tenantId;
    }

    public void ChangeTenant(
        long tenantId)
    {
        // Invalid:
        // _tenantId = tenantId;
    }
}
```

`readonly` protects against reassignment after initialization.

✦ readonly Is Runtime Initialization

Unlike `const`, `readonly` can use runtime values.

Example:

```C#
private readonly Guid _id =
    Guid.NewGuid();
```

This is valid because:

EXAMPLE / FLOW

`readonly`

does not require compile-time evaluation.

✦ readonly with DateTime

```C#
private readonly DateTime _createdAtUtc =
    DateTime.UtcNow;
```

Valid.

But:

```C#
const DateTime CreatedAtUtc =
    DateTime.UtcNow;
```

is invalid.

✦ readonly Reference Type Important Rule

This is one of the most important interview points.

Suppose:

```C#
private readonly List<long> _giftIds =
    new();
```

Does `readonly` make the list immutable?

No.

This is valid:

```C#
_giftIds.Add(
    1001);
```

This is valid:

```C#
_giftIds.Remove(
    1001);
```

But this is not valid after initialization:

```C#
// _giftIds = new List<long>();
```

✦ readonly Reference Means

EXAMPLE / FLOW

```
The reference cannot be reassigned.
```

It does **not** mean:

EXAMPLE / FLOW

```
The referenced object cannot change.
```

✦ readonly Reference Flow

EXAMPLE / FLOW

```
_readonly field
    →
Reference
    →
List Object
```

Protected:

EXAMPLE / FLOW

```
Cannot redirect field
to another List object
```

Not protected:

EXAMPLE / FLOW

```
List object's internal contents
can still change
```

✦ Example

C#

```
public sealed class GiftBatch
{
    private readonly List<long>
        _giftIds = new();

    public void AddGift(
        long giftId)
    {
        _giftIds.Add(
            giftId);
    }
}
```

This is perfectly valid.

The list reference is stable.

The list content is mutable.

✦ readonly Is Not Immutability

Do not say:

☀ readonly makes an object immutable.

That is incorrect.

More accurate:

☀ `readonly` prevents reassignment of the field outside its allowed initialization contexts.

The referenced object's state may still be mutable.

✦ Achieving Stronger Immutability

You may need:

EXAMPLE / FLOW

```
Immutable object design

get-only properties

init properties

readonly structs

immutable collections

defensive copies
```

depending on the requirement.

Example:

C#

```
public sealed class GiftConfiguration
{
    public GiftConfiguration(
        string currencyCode)
    {
        CurrencyCode =
            currencyCode;
    }

    public string CurrencyCode
    {
        get;
    }
}
```

No setter exists.

This is stronger than merely making a private field reference readonly if the underlying object is mutable.

✦ readonly Value Type Field

Example:

C#

```
private readonly decimal
    _minimumGiftAmount;
```

Because `decimal` is a value type:

EXAMPLE / FLOW

```
readonly
```

prevents changing that field value after initialization.

There is no separate mutable object behind the decimal value.

✦ readonly Struct Preview

You may also see:

```C#
public readonly struct Money
{
}
```

A `readonly struct` expresses that the struct's instance state should not be modified after creation. Current C# rules require its instance data members to respect readonly behavior.

This is different from:

```C#
private readonly Money _money;
```

One describes the type.

The other describes a field.

✦ What Is static readonly?

Now combine:

```C#
static
+
readonly
```

Example:

```C#
public static readonly Guid
    ApplicationInstanceId =
        Guid.NewGuid();
```

Meaning:

```
C#
static
    →
one field shared by the type

readonly
    →
can be initialized once
then not reassigned
```

✦ static readonly Initialization

A `static readonly` field can normally be assigned:

```
C#
At its declaration

or

Inside the type's static constructor
```

The language specification explicitly permits those contexts.

✦ Example

```
C#
public static readonly DateTime
    StartedAtUtc =
        DateTime.UtcNow;
```

This value:

```
EXAMPLE / FLOW
is calculated at runtime
```

and shared by the whole type.

✦ Static Constructor Assignment

```
C#
public static class ApplicationInfo
{
    public static readonly DateTime
        StartedAtUtc;

    static ApplicationInfo()
    {
        StartedAtUtc =
            DateTime.UtcNow;
    }
}
```

Valid.

✦ const vs readonly vs static readonly

This is one of the most important comparison tables in the topic.

Feature	`const`	`readonly`	`static readonly`
Known at compile time	Yes	Not required	Not required
Runtime initialization allowed	No	Yes	Yes
Per object	No	Usually yes	No
Shared by type	Yes	No unless static	Yes
Assigned in constructor	No	Instance constructor	Static constructor
Reassigned after initialization	No	No	No
Can hold runtime-created object	No	Yes	Yes
Public library value versioning	Value may be embedded	Runtime field lookup	Runtime field lookup

✦ Simple Mental Model

```C#
const
    →
Fixed forever at compile time

readonly
    →
Fixed after this object's construction

static readonly
    →
Fixed after type initialization
and shared by all instances
```

✦ Example Together

C#

```
public sealed class GiftProcessor
{
    public const int DefaultBatchSize =
        100;

    private readonly long _tenantId;

    public static readonly DateTime
        ApplicationStartUtc =
            DateTime.UtcNow;

    public GiftProcessor(
        long tenantId)
    {
        _tenantId =
            tenantId;
    }
}
```

Interpretation:

EXAMPLE / FLOW

```
DefaultBatchSize
    →
compile-time constant

_tenantId
    →
different per GiftProcessor object
but fixed after construction

ApplicationStartUtc
    →
one shared runtime-created value
for the type
```

✦ What Is static?

Definition

`static` means the member belongs to the **type itself** rather than to a particular object instance.

Example:

C#

```
public static int GiftCount;
```

Access:

```C#
Gift.GiftCount
```

rather than:

```C#
gift.GiftCount
```

Static members are accessed through the type name, and there is one shared static member for the relevant type rather than one per object.

✦ Instance Member

Example:

```C#
public decimal TotalAmount
{
    get;
    set;
}
```

You need an object:

```C#
var gift =
    new Gift();

gift.TotalAmount =
    1000m;
```

✦ Static Member

Example:

```C#
public static int CreatedGiftCount
{
    get;
    private set;
}
```

Access:

```C#
int count =
    Gift.CreatedGiftCount;
```

No `Gift` instance is required.

✦ Instance vs Static Flow

EXAMPLE / FLOW

```
Instance Member

Gift Object
    →
TotalAmount


Static Member

Gift Type
    →
CreatedGiftCount
```

✦ Static Fields

Example:

C#

```
public sealed class Gift
{
    private static int _createdCount;

    public Gift()
    {
        _createdCount++;
    }

    public static int CreatedCount
        =>
        _createdCount;
}
```

Usage:

C#

```
var gift1 =
    new Gift();

var gift2 =
    new Gift();

Console.WriteLine(
    Gift.CreatedCount);
```

Result:

EXAMPLE / FLOW

```
2
```

✦ Static State Is Shared State

This is one of the most important practical rules.

If several objects access:

```C#
Gift.CreatedCount
```

they all use the same state.

Conceptually:

```C#
Request A ──┐
            ↓
       static field
            ↑
Request B ──┘
```

This can create concurrency issues when the field is mutable.

✦ Static Properties

A property can also be static.

Example:

```C#
public static string ApplicationName
    =>
    "Fundraising Platform";
```

Usage:

```C#
string name =
    ApplicationInfo.ApplicationName;
```

✦ Static Methods

A static method belongs to the type.

Example:

```C#
public static decimal CalculateFee(
    decimal giftAmount,
    decimal rate)
{
    return giftAmount * rate;
}
```

Call:

```C#
decimal fee =
    GiftCalculator.CalculateFee(
        1000m,
        0.02m);
```

No object instance is required.

✦ Static Method Cannot Directly Access Instance State

Example:

```C#
public sealed class Gift
{
    public decimal TotalAmount
    {
        get;
        set;
    }

    public static void PrintAmount()
    {
        // Invalid:
        // Console.WriteLine(TotalAmount);
    }
}
```

Why?

`TotalAmount` belongs to:

EXAMPLE / FLOW

```
a particular Gift object
```

but static method has no automatic `this` instance.

✦ Passing Instance Explicitly

A static method can still use instance data if you pass an object.

```C#
public static void PrintAmount(
    Gift gift)
{
    Console.WriteLine(
        gift.TotalAmount);
}
```

Now the method receives the object explicitly.

✦ static vs Instance Method

Static Method	Instance Method
Belongs to type	Belongs to object
No instance required	Requires object
No implicit `this`	Has current object instance
Cannot directly use instance fields	Can use instance fields
Often stateless utility/calculation	Often object/domain behavior

✦ When Static Methods Are Good

Use static methods for operations that:

EXAMPLE / FLOW

```
Do not depend on object identity

Do not depend on mutable instance state

Are naturally utility or transformation operations
```

Example:

C#

```
public static decimal CalculatePercentage(
    decimal value,
    decimal total)
{
    return total == 0m
        ? 0m
        : value / total * 100m;
}
```

✦ When Static Methods Are Not Ideal

Avoid turning all services into static methods when the operation needs:

EXAMPLE / FLOW

```
Dependencies

Testing/mocking boundaries

Configuration

Database access

HttpClient

Logging

Per-request state
```

For example, avoid:

C#

```
public static class GiftService
{
    public static void CreateGift()
    {
        // database
        // logging
        // email
        // payment
    }
}
```

In ASP.NET Core, these responsibilities are normally better represented by injectable services.

✦ What Is a Static Class?

Definition

A static class:

C#

```
Contains only static members

Cannot be instantiated

Cannot be inherited from as a normal base class

Is implicitly sealed
```

Microsoft documents these as core static-class properties.

✦ Example

```C#
public static class GiftMath
{
    public static decimal CalculateFee(
        decimal amount,
        decimal rate)
    {
        return amount * rate;
    }
}
```

Use:

```C#
decimal fee =
    GiftMath.CalculateFee(
        1000m,
        0.02m);
```

You cannot do:

```C#
// var math = new GiftMath();
```

✦ Good Static Class Examples

Good uses:

```EXAMPLE / FLOW
Pure utilities

Extension methods

Stateless calculations

Constants

Simple mappings
```

Examples from .NET include:

```EXAMPLE / FLOW
Math

Console
```

✦ Bad Static Class Example

Avoid using a static class merely as a global storage container:

```C#
public static class GlobalData
{
    public static Gift CurrentGift;

    public static User CurrentUser;

    public static DbConnection Connection;

    public static List<Gift> Gifts;
}
```

Problems:

EXAMPLE / FLOW

```
Shared mutable state

Concurrency issues

Testing difficulty

Hidden dependencies

Memory lifetime issues

Cross-request data leakage
```

This becomes particularly dangerous in web applications.

✦ Static Constructors

Definition

A **static constructor** initializes static state or performs initialization that should happen once for a type.

Syntax:

```C#
public sealed class GiftRules
{
    static GiftRules()
    {
    }
}
```

Notice:

```C#
No access modifier

No parameters

No return type
```

The runtime calls it automatically.

Microsoft documents that a static constructor runs automatically before the type's first relevant use and is called at most once.

✦ Static Constructor Example

C#

```
public sealed class GiftConfiguration
{
    public static readonly DateTime
        InitializedAtUtc;

    static GiftConfiguration()
    {
        InitializedAtUtc =
            DateTime.UtcNow;
    }
}
```

✦ Static Constructor Rules

A static constructor:

EXAMPLE / FLOW

```
Has no access modifier

Has no parameters

Cannot be called manually

Cannot be inherited

Cannot be overloaded

Runs at most once per type initialization context
```

The CLR invokes it automatically.

✦ Static Constructor vs Instance Constructor

Instance constructor:

C#

```
public Gift()
{
}
```

runs when:

EXAMPLE / FLOW

```
a Gift object is created.
```

Static constructor:

C#

```
static Gift()
{
}
```

runs for:

EXAMPLE / FLOW

```
type initialization.
```

✦ Flow

EXAMPLE / FLOW

```
First Relevant Gift Type Use
    →
Static Initialization
    →
Static Constructor
    →
Then Instance Construction
```

For ordinary type usage, the static constructor runs before instance construction requiring that type to be initialized.

✦ Example

C#

```
public sealed class Gift
{
    static Gift()
    {
        Console.WriteLine(
            "Static constructor");
    }

    public Gift()
    {
        Console.WriteLine(
            "Instance constructor");
    }
}
```

Create:

```C#
var gift1 =
    new Gift();

var gift2 =
    new Gift();
```

Conceptual output:

EXAMPLE / FLOW

```
Static constructor

Instance constructor

Instance constructor
```

Static initialization occurs once.

Instance construction occurs once per object.

✦ Static Constructor Exception

This is a serious issue.

If a static constructor throws an unhandled exception, the runtime does not simply retry it every time.

The type can remain unusable for that runtime lifetime, commonly surfacing through `TypeInitializationException`.

Therefore avoid:

EXAMPLE / FLOW

```
Network calls

Database calls

Blocking asynchronous operations

Complex external dependencies
```

inside static constructors.

✦ Bad Static Constructor

```C#
static GiftConfiguration()
{
    Configuration =
        LoadFromRemoteApi();
}
```

Potential problems:

EXAMPLE / FLOW

```
Network unavailable

Timeout

Exception

Type becomes unusable
```

This is usually poor application design.

✦ Static Constructor Thread Safety

The CLR performs type initialization in a synchronized manner.

You do not normally need an additional lock merely to make a static constructor execute once.

However, Microsoft warns against blocking or complex synchronization inside static constructors because deadlocks can occur.

✦ Prefer Inline Static Initialization When Simple

Instead of:

C#

```
public static readonly Guid
    ApplicationId;

static ApplicationInfo()
{
    ApplicationId =
        Guid.NewGuid();
}
```

you can often write:

C#

```
public static readonly Guid
    ApplicationId =
        Guid.NewGuid();
```

For simple static field initialization, inline initialization is generally clearer and may allow better runtime optimization than an explicit static constructor.

✦ Initialization Order — Simplified Beginner Model

For an object:

EXAMPLE / FLOW

```
Type initialization if needed
    →
Instance field initialization
    →
Base/derived constructor flow
    →
Instance constructor
    →
Object ready
```

The exact C# initialization sequence has detailed inheritance rules, which will be studied more deeply in the constructors topic. Microsoft documents that static construction occurs before instance-construction actions for a type and that instance initialization follows defined base/derived sequencing.

For this topic remember:

EXAMPLE / FLOW

```
Static initialization
    →
once for type

Instance initialization
    →
for every object
```

✦ Object Lifetime

Now we connect these keywords to lifetime.

Definition

An object's lifetime is roughly:

EXAMPLE / FLOW

```
Object Created
    →
Object Used
    →
Object No Longer Reachable
    →
Eventually Eligible for Garbage Collection
```

C# uses automatic memory management rather than requiring normal managed objects to be manually freed.

✦ Object Creation

Example:

```
C#
Gift gift =
    new Gift();
```

Conceptually:

```
EXAMPLE / FLOW
new Gift()
    →
Managed object created
    →
Constructor runs
    →
Reference stored in gift
```

The object is alive while the runtime can still reach it through relevant references.

✦ Reference Lifetime vs Object Lifetime

These are not always identical.

Example:

```
C#
Gift gift =
    new Gift();
```

Later:

```
C#
gift =
    null;
```

The variable stopped referencing the object.

But the object might still be alive if another reference exists.

Example:

```
C#
Gift gift1 =
    new Gift();

Gift gift2 =
    gift1;

gift1 =
    null;
```

The object is still reachable through:

```
EXAMPLE / FLOW
gift2
```

so it is not garbage merely because `gift1` became null.

✦ Multiple References

EXAMPLE / FLOW

```
gift1 ──┐
        ↓
     Gift Object
        ↑
gift2 ──┘
```

Then:

EXAMPLE / FLOW

```
gift1 = null
```

leaves:

EXAMPLE / FLOW

```
gift2
   →
Gift Object
```

Object is still reachable.

✦ When Does an Object Become Eligible for GC?

Simplified:

EXAMPLE / FLOW

```
No live root
can reach the object
   →
Object becomes unreachable
   →
GC may reclaim it later
```

The garbage collector determines reachability from GC roots. Microsoft lists roots such as static fields, active thread stack references, GC handles, and finalization-related structures.

✦ Important: Eligible Does Not Mean Immediately Collected

If an object becomes unreachable:

EXAMPLE / FLOW

```
it does not mean
memory disappears immediately.
```

It means:

EXAMPLE / FLOW

```
GC can reclaim it
during a future collection.
```

You generally do not control the exact moment.

✦ Static Fields and Object Lifetime

This is extremely important.

Suppose:

C#

```
public static Gift? CurrentGift;
```

Then:

C#

```
GiftState.CurrentGift =
    new Gift();
```

Conceptually:

EXAMPLE / FLOW

```
Static Field
    →
Gift Object
```

A static field is a GC root or reachable from static root data.

As long as that static reference remains alive:

EXAMPLE / FLOW

```
the Gift object remains reachable
```

and normally cannot be collected.

Microsoft explicitly lists static fields/static data among GC roots used to determine object liveness.

✦ Static Lifetime Problem

Consider:

C#

```
public static class GiftCache
{
    public static readonly List<Gift>
        Gifts = new();
}
```

Every added gift:

```C#
GiftCache.Gifts.Add(
    gift);
```

remains reachable while the list keeps its reference.

Flow:

```EXAMPLE / FLOW
Static GiftCache
    →
Static List
    →
Gift 1
Gift 2
Gift 3
Gift 4
...
```

If you continuously add objects and never remove them:

```EXAMPLE / FLOW
memory usage can keep growing.
```

This is a real memory-retention problem.

✦ GC Does Not Save You From Reachable Memory

A common misunderstanding:

☀ .NET has garbage collection, so memory leaks cannot happen.

Incorrect.

If your program unintentionally keeps objects reachable:

```EXAMPLE / FLOW
GC correctly sees them as live
```

and does not collect them.

Example:

```C#
static collection
    →
keeps millions of objects
```

The GC is working correctly.

Your reference lifetime is wrong.

✦ Managed Memory Leak Concept

In managed .NET applications, a common "memory leak" is really:

EXAMPLE / FLOW

```
Objects no longer needed
    →
but still reachable
    →
GC cannot reclaim
```

Static references are a common cause.

✦ Static Event References Preview

Another common lifetime issue:

C#

```
static publisher
    →
event delegate
    →
subscriber object
```

If subscriber never unsubscribes, a long-lived static publisher can keep subscriber objects alive.

We will study this carefully in the Events topic.

✦ Object Lifetime and Local Variables

Example:

C#

```
void Process()
{
    Gift gift =
        new Gift();

    ProcessGift(
        gift);
}
```

After the method finishes:

EXAMPLE / FLOW

```
gift local
```

no longer provides an active reference.

If no other references remain:

EXAMPLE / FLOW

```
Gift object
    →
becomes eligible for collection
```

The exact physical lifetime can be optimized by the JIT/GC; scope does not necessarily define the exact instant at which the object is considered dead. The language specification explicitly allows runtime analysis to determine that an object is no longer needed even before a local variable's textual scope ends.

✦ Scope vs Lifetime

Do not confuse:

EXAMPLE / FLOW

```
scope
```

with:

EXAMPLE / FLOW

```
object lifetime
```

Scope means:

EXAMPLE / FLOW

```
Where a variable name can be used in source code.
```

Lifetime means:

EXAMPLE / FLOW

```
How long the runtime object remains reachable/alive.
```

✦ Example

C#

```
void Process()
{
    Gift gift =
        new Gift();

    cache.Add(
        gift);
}
```

When method ends:

EXAMPLE / FLOW

```
local gift variable is gone
```

but:

EXAMPLE / FLOW

```
cache
```

still references the object.

Therefore the object remains alive.

✦ Static Member Lifetime

Static state typically lives for the lifetime of the type's loaded runtime context.

For ordinary application code, that often effectively means:

EXAMPLE / FLOW

```
from type initialization
until application/process/load-context teardown
```

Microsoft documentation describes static classes/static state as remaining for the lifetime of the application domain/context in which they are loaded.

For practical ASP.NET Core thinking:

C#

```
static mutable state
can live a very long time
```

often as long as the application process.

✦ Instance Lifetime

Instance object lifetime depends on:

EXAMPLE / FLOW

```
references
```

not simply on:

C#

```
class definition
```

Each object can die independently when no reachable references remain.

Example:

EXAMPLE / FLOW

```
Gift Object A
    →
no references
    →
eligible for GC

Gift Object B
    →
still referenced
    →
remains alive
```

✦ static Does Not Mean Thread-Safe

This is a critical interview rule.

Suppose:

C#

```
public static int GiftCount;
```

Several requests run concurrently:

C#

```
GiftCount++;
```

This is not automatically thread-safe.

Why?

`++` conceptually involves:

EXAMPLE / FLOW

```
Read current value
    →
Add 1
    →
Write result
```

Two threads can interleave these operations.

✦ Race Condition Example

Initial:

EXAMPLE / FLOW

```
GiftCount = 10
```

Thread A:

EXAMPLE / FLOW

```
reads 10
```

Thread B:

EXAMPLE / FLOW

```
reads 10
```

Thread A:

EXAMPLE / FLOW

```
writes 11
```

Thread B:

EXAMPLE / FLOW

```
writes 11
```

Expected:

EXAMPLE / FLOW

```
12
```

Actual:

EXAMPLE / FLOW

```
11
```

This is a race condition.

✦ Thread-Safe Counter

Later we will study:

C#

```
Interlocked.Increment(
    ref _giftCount);
```

which performs atomic increment semantics.

But the important rule now is:

C#

```
static
≠
thread-safe
```

✦ Static readonly Is Not Automatically Thread-Safe Either

Consider:

```C#
public static readonly List<Gift>
    Gifts = new();
```

The reference cannot change.

But the list is mutable.

Multiple threads calling:

```C#
Gifts.Add(...)
```

can still cause concurrency problems.

Remember:

```EXAMPLE / FLOW
readonly reference
≠
immutable object
≠
thread-safe object
```

✦ Important Three-Way Distinction

```EXAMPLE / FLOW
readonly
    →
Cannot reassign field

immutable
    →
Object state cannot change

thread-safe
    →
Object can be safely used concurrently
according to its contract
```

These are three different concepts.

✦ Example

```C#
public static readonly List<long>
    GiftIds = new();
```

Properties:

EXAMPLE / FLOW

```
Reference reassignment
    →
Prevented

List mutation
    →
Allowed

Concurrent mutation safety
    →
Not guaranteed
```

✦ Better Shared Read-Only Data

If data truly never changes:

C#

```
public static readonly IReadOnlyList<string>
    SupportedCurrencies =
        new[]
        {
            "USD",
            "EUR"
        };
```

But note:

EXAMPLE / FLOW

```
IReadOnlyList interface
```

alone does not always guarantee the underlying object can never change elsewhere.

For real immutability, immutable collections or private ownership may be needed.

✦ static and ASP.NET Core

This is where practical full-stack backend knowledge matters.

ASP.NET Core already has a dependency-injection container.

Do not use static state as a replacement for:

EXAMPLE / FLOW

```
Singleton services

Scoped services

Configuration

Caching abstractions

Repositories

DbContext

HttpClientFactory
```

✦ Bad ASP.NET Core Example

C#

```
public static class CurrentRequest
{
    public static long UserId;

    public static long TenantId;
}
```

This is extremely dangerous in a web server.

Multiple requests execute concurrently.

Request A may set:

EXAMPLE / FLOW

```
TenantId = 1
```

while Request B sets:

EXAMPLE / FLOW

```
TenantId = 2
```

Now shared static state can leak between requests.

✦ Request Flow Problem

EXAMPLE / FLOW

```
Request A
Tenant 1
    ──┐
      ↓
Static TenantId
      ↑
Request B
Tenant 2
```

One global value cannot safely represent two concurrent request contexts.

✦ Correct Direction

Use request/scoped abstractions:

EXAMPLE / FLOW

```
HTTP Request
    →
Scoped User/Tenant Context
    →
Request-specific state
```

Later Dependency Injection and ASP.NET Core topics will explain this in detail.

✦ Static DbContext Is Dangerous

Never design:

C#

```
public static DbContext Context;
```

for a normal ASP.NET Core application.

EF Core `DbContext` has scoped/unit-of-work behavior and is not intended as globally shared static mutable state across requests.

Later EF Core topics will explain why.

✦ Static HttpClient Nuance

You may hear:

C#

```
HttpClient should be static.
```

This is oversimplified.

Historically long-lived `HttpClient` was recommended to avoid socket exhaustion, but modern ASP.NET Core commonly uses:

EXAMPLE / FLOW

```
IHttpClientFactory
```

for managed handler lifetimes, configuration, resilience, and DI integration.

Therefore do not blindly turn external dependencies into static fields.

✦ Static Utility Is Fine

This is perfectly reasonable:

C#

```
public static class GiftMath
{
    public static decimal CalculatePercentage(
        decimal value,
        decimal total)
    {
        return total == 0m
            ? 0m
            : value / total * 100m;
    }
}
```

Why?

EXAMPLE / FLOW

```
No shared mutable state

No external dependency

Pure calculation
```

✦ Fundraising Example — const

C#

```
public static class GiftRules
{
    public const decimal MinimumGiftAmount =
        0.01m;

    public const int CurrencyCodeLength =
        3;
}
```

Use:

```C#
if (giftAmount <
    GiftRules.MinimumGiftAmount)
{
    throw new ArgumentOutOfRangeException(
        nameof(giftAmount));
}
```

✦ Fundraising Example — readonly

```C#
public sealed class GiftService
{
    private readonly long _tenantId;

    public GiftService(
        long tenantId)
    {
        _tenantId =
            tenantId;
    }
}
```

Each service object can have:

EXAMPLE / FLOW

```
different tenantId
```

but its field cannot later be redirected to another tenant value.

✦ Better Real ASP.NET Example

Later DI will normally inject dependencies:

```C#
public sealed class GiftService
{
    private readonly IGiftRepository
        _giftRepository;

    private readonly ILogger<GiftService>
        _logger;

    public GiftService(
        IGiftRepository giftRepository,
        ILogger<GiftService> logger)
    {
        _giftRepository =
            giftRepository;

        _logger =
            logger;
    }
}
```

Why readonly?

EXAMPLE / FLOW

```
Dependencies assigned in constructor
    →
Service keeps same dependency references
throughout object lifetime
```

This is one of the most common professional uses of `readonly`.

✦ Dependency Injection Flow

EXAMPLE / FLOW

```
DI Container
    →
Creates GiftService
    →
Injects Repository
    →
Injects Logger
    →
Constructor assigns readonly fields
    →
Service uses same dependencies
for its lifetime
```

✦ static readonly Example

C#

```
public static class GiftFormats
{
    public static readonly string[]
        SupportedCurrencies =
        {
            "USD",
            "EUR",
            "GBP"
        };
}
```

However, remember:

EXAMPLE / FLOW

```
readonly array reference
```

does not make array contents immutable.

This is still possible:

```C#
GiftFormats.SupportedCurrencies[0] =
    "ABC";
```

So this design is not fully immutable.

✦ Better Immutable Intent

Depending on need, expose:

```C#
public static IReadOnlyList<string>
    SupportedCurrencies
{
    get;
} =
    new[]
    {
        "USD",
        "EUR",
        "GBP"
    };
```

Or use an immutable collection where actual immutability is important.

✦ static readonly Object Example

```C#
public static readonly TimeSpan
    DefaultTimeout =
        TimeSpan.FromSeconds(
            30);
```

This is an excellent example.

Why not `const`?

Because:

```C#
TimeSpan.FromSeconds(30)
```

requires runtime construction.

Why `static readonly`?

EXAMPLE / FLOW

```
One shared value
+
initialized once
```

✦ const vs static readonly Example

Compile-time:

```C#
public const int DefaultPageSize =
    50;
```

Runtime-created:

```C#
public static readonly TimeSpan
    DefaultTimeout =
        TimeSpan.FromSeconds(
            30);
```

✦ Configuration Should Usually Not Be const

Bad:

```C#
public const string PaymentApiUrl =
    "https://api.example.com";
```

Why?

Endpoint may differ across:

```EXAMPLE / FLOW
Development

QA

Production
```

Better:

```EXAMPLE / FLOW
Configuration
    →
appsettings / environment
    →
Options pattern
```

We will study this in ASP.NET Core configuration.

✦ static readonly Is Still Not Configuration

Do not simply replace:

```C#
const string ApiUrl
```

with:

```C#
static readonly string ApiUrl
```

if the actual value should be externally configurable.

The real question is:

EXAMPLE / FLOW

```
Is this a code invariant
or
runtime configuration?
```

✦ Decision Flow

```C#
Value Needed
    →
Must it vary by environment/runtime?
    →
Yes
    →
Configuration / Options

No
    →
Known at compile time?
    →
Yes
    →
const may fit

No
    →
One shared runtime value?
    →
Yes
    →
static readonly may fit

Per-object runtime value?
    →
readonly may fit
```

✦ Object Lifetime and Dependency Lifetimes Preview

Later ASP.NET Core DI introduces:

EXAMPLE / FLOW

```
Transient

Scoped

Singleton
```

Those are service lifetimes.

They are different from C# keywords:

C#

```
static

readonly

const
```

But the concepts interact.

✦ Transient Preview

EXAMPLE / FLOW

```
New object
for each resolution/use
```

✦ Scoped Preview

EXAMPLE / FLOW

```
One object
per DI scope
often per HTTP request
```

✦ Singleton Preview

EXAMPLE / FLOW

```
One object
for application container lifetime
```

✦ static vs Singleton

These are not identical.

Static:

EXAMPLE / FLOW

```
Global type-level member
No instance
No normal interface-based DI lifecycle
```

Singleton service:

EXAMPLE / FLOW

```
One object instance
Managed by DI
Can implement interfaces
Can receive dependencies
Can be replaced/mocked more naturally
```

Do not say:

C#

```
Singleton = static
```

They solve related but different problems.

✦ Fundraising Example

Static utility:

C#

```
GiftMath.CalculatePercentage(...)
```

Singleton service:

EXAMPLE / FLOW

```
ICurrencyMetadataProvider
    →
one application-wide instance
```

Scoped service:

EXAMPLE / FLOW

```
GiftService
    →
one per request
```

These are different design decisions.

✦ Object Lifetime and IDisposable Preview

Garbage collection manages:

EXAMPLE / FLOW

```
managed memory
```

but does not mean every resource can simply wait for GC.

Resources such as:

EXAMPLE / FLOW

```
Files

Sockets

Database connections

Native handles
```

may require deterministic cleanup.

Later we will study:

EXAMPLE / FLOW

```
IDisposable

Dispose

using

IAsyncDisposable
```

under Resource Management.

✦ GC Lifetime vs Resource Lifetime

Important:

EXAMPLE / FLOW

```
Object memory lifetime
```

and:

EXAMPLE / FLOW

```
external resource lifetime
```

are related but not identical.

Example:

EXAMPLE / FLOW

```
FileStream object
    →
managed object

Operating-system file handle
    →
external resource
```

The file handle often should be released promptly through disposal rather than waiting for GC.

✦ Common Mistake — const for Runtime Values

Invalid:

C#

```
const DateTime StartedAt =
    DateTime.UtcNow;
```

Use:

C#

```
static readonly DateTime StartedAt =
    DateTime.UtcNow;
```

if one shared runtime timestamp is appropriate.

✦ Common Mistake — readonly Means Constant

Not exactly.

`readonly` can differ for every object.

Example:

C#

```
private readonly long _tenantId;
```

GiftService A:

EXAMPLE / FLOW

```
Tenant 1
```

GiftService B:

EXAMPLE / FLOW

```
Tenant 2
```

Both fields are readonly, but values differ.

✦ Common Mistake — readonly Makes Object Immutable

Wrong:

```C#
private readonly List<Gift>
    _gifts = new();
```

You can still:

```C#
_gifts.Add(
    gift);
```

Readonly protects the field reference, not necessarily the object's internals.

✦ Common Mistake — static Means Constant

Incorrect.

```C#
public static int Count;
```

can change.

`static` means:

EXAMPLE / FLOW

```
shared/type-level
```

not:

EXAMPLE / FLOW

```
unchangeable
```

✦ Common Mistake — static Means Thread-Safe

Incorrect.

```C#
public static int Count;
```

is shared, which actually makes thread-safety concerns more important.

✦ Common Mistake — static readonly Means Immutable

Incorrect.

```C#
public static readonly List<int>
    Values = new();
```

The field reference is fixed.

The list remains mutable.

✦ Common Mistake — Public Mutable Static Fields

Avoid:

```C#
public static List<Gift>
    Gifts = new();
```

Anyone can:

EXAMPLE / FLOW

```
replace list

change list

clear list

mutate it concurrently
```

This is uncontrolled global state.

✦ Common Mistake — Using static Instead of Dependency Injection

Bad:

```C#
DatabaseHelper.SaveGift(...);
```

where `DatabaseHelper` internally owns global mutable database state.

This creates:

EXAMPLE / FLOW

```
Hidden dependencies

Hard testing

Global coupling

Concurrency risk
```

Later DI topics will show better patterns.

✦ Common Mistake — static Cache Without Eviction

Example:

```C#
static Dictionary<long, Gift>
    Cache = new();
```

If entries continuously accumulate:

```C#
static root
    →
dictionary
    →
gifts remain reachable
```

memory can keep growing.

Caches need policies such as:

EXAMPLE / FLOW

```
expiration

size limits

eviction
```

where appropriate.

✦ Common Mistake — Config Values as const

Avoid:

```C#
public const int ApiTimeoutSeconds =
    30;
```

if operations teams should be able to change it without recompiling.

Use runtime configuration.

✦ Common Mistake — Changing Public const in Library

A public const value may be compiled into consuming assemblies.

Changing the library without rebuilding consumers can create inconsistent behavior.

Use `static readonly` when runtime lookup/versioning behavior is preferable.

✦ Common Mistake — Heavy Static Constructor

Avoid:

EXAMPLE / FLOW

```
Database connection

Network request

File download

Task.Wait()

Thread synchronization

long-running initialization
```

inside static constructors.

Static initialization failure can make the type unusable, and blocking operations can create deadlock or startup problems.

✦ Performance Considerations

const

Because constants are known at compile time:

EXAMPLE / FLOW

```
compiler can substitute value directly.
```

There is generally no runtime field lookup requirement for ordinary constant usage.

This is usually not something you need to manually optimize around.

✦ static readonly

A `static readonly` value is obtained from static runtime storage.

Its value is not necessarily known at compile time.

This enables runtime initialization and different versioning semantics.

✦ readonly Instance Fields

Readonly fields can help express stable object structure.

Example:

C#

```
private readonly IGiftRepository
    _repository;
```

This improves:

EXAMPLE / FLOW

```
Correctness

Reasoning

Object invariants
```

The performance difference is usually not the main reason to use it.

✦ static Memory Retention

Static references can keep large object graphs alive.

Example:

EXAMPLE / FLOW

```
Static Dictionary
    →
Tenant
    →
Constituents
    →
Gifts
    →
Addresses
```

If the dictionary remains rooted:

EXAMPLE / FLOW

```
the entire reachable graph
may remain alive.
```

Always consider lifetime when putting large objects into static fields.

✦ Static Generic Types — Advanced Point

Suppose:

C#

```
public static class Cache<T>
{
    public static int Count;
}
```

Then:

EXAMPLE / FLOW

```
Cache<int>.Count

Cache<string>.Count
```

are separate static fields because each closed generic type has its own static state. The language specification explicitly distinguishes static state per closed constructed type.

✦ Example

C#

```
Cache<int>.Count =
    5;

Cache<string>.Count =
    10;
```

These are independent.

This can be useful but can also surprise developers.

✦ Static Local Variables

C# does not support C/C++-style static local variables inside methods.

This is invalid conceptually:

C#

```
void Process()
{
    // static int count;
}
```

Microsoft's static member documentation explicitly notes that C# does not support static local variables.

If shared state is required, it belongs elsewhere in the type/design.

✦ Security Considerations

Static mutable state can become a security risk.

Imagine:

C#

```
public static long CurrentTenantId;
```

In a multi-tenant web application:

EXAMPLE / FLOW

```
Request A
    →
Tenant 100

Request B
    →
Tenant 200
```

If both requests share:

EXAMPLE / FLOW

```
CurrentTenantId
```

one tenant's value can overwrite the other's.

This can potentially lead to:

EXAMPLE / FLOW

```
Cross-tenant data access

Data leakage

Incorrect authorization
```

Never store request-specific tenant/user identity in ordinary mutable static fields.

✦ Secrets and static Fields

Avoid keeping sensitive mutable values in public static state.

Example:

C#

```
public static string ApiSecret;
```

Problems include:

EXAMPLE / FLOW

```
Global accessibility

Hard rotation

Accidental logging

Long lifetime
```

Use secure configuration/secrets abstractions and limit access.

✦ Thread Safety of Static Collections

Bad:

```C#
public static readonly Dictionary<long, Gift>
    Gifts = new();
```

Concurrent writes can be unsafe.

Alternatives depend on the requirement:

EXAMPLE / FLOW

```
ConcurrentDictionary

lock

immutable snapshot

cache abstraction

database
```

Later concurrency topics will cover this properly.

✦ Practical Exercise — const

Create a constant for maximum retry count.

```C#
public const int MaxRetryCount =
    3;
```

Question:

Can this later be assigned?

```C#
// MaxRetryCount = 5;
```

Answer:

EXAMPLE / FLOW

```
No.
```

✦ Practical Exercise — readonly

```C#
public sealed class GiftService
{
    private readonly long _tenantId;

    public GiftService(
        long tenantId)
    {
        _tenantId =
            tenantId;
    }
}
```

Can another method do:

```C#
_tenantId =
    500;
```

Answer:

EXAMPLE / FLOW

No.

✦ Practical Exercise — readonly Reference

```C#
private readonly List<long>
    _giftIds = new();
```

Which is valid?

```C#
_giftIds.Add(
    100);
```

Answer:

EXAMPLE / FLOW

Valid.

What about:

```C#
_giftIds =
    new List<long>();
```

after construction?

Answer:

EXAMPLE / FLOW

```
Not valid.
```

✦ Practical Exercise — static

```C#
public sealed class Gift
{
    public static int Count;
}
```

Create:

```C#
Gift.Count =
    10;
```

Then create 100 Gift objects.

How many `Count` fields exist for the type?

Conceptually:

```C#
One shared static field.
```

✦ Practical Exercise — Instance Field

```C#
public sealed class Gift
{
    public decimal Amount;
}
```

Create:

```C#
var gift1 =
    new Gift
    {
        Amount = 100m
    };

var gift2 =
    new Gift
    {
        Amount = 500m
    };
```

Results:

EXAMPLE / FLOW

```
gift1.Amount = 100

gift2.Amount = 500
```

Each object has its own instance field.

✦ Practical Exercise — static readonly

C#

```
public static readonly DateTime
    StartedAtUtc =
        DateTime.UtcNow;
```

Why not `const`?

Answer:

EXAMPLE / FLOW

```
DateTime.UtcNow
is determined at runtime,
not compile time.
```

✦ Practical Exercise — const Versioning

Library:

C#

```
public const int Limit =
    10;
```

Consumer compiles.

Later library changes:

C#

```
public const int Limit =
    20;
```

Consumer assembly is not rebuilt.

Can consumer potentially continue using `10`?

Answer:

EXAMPLE / FLOW

```
Yes.
```

Because the constant value may have been compiled into the caller.

✦ Practical Exercise — Static Reference Lifetime

```C#
public static class Cache
{
    public static Gift? Gift;
}
```

Then:

```C#
Cache.Gift =
    new Gift();
```

If no other code uses the gift but `Cache.Gift` still references it, is it normally eligible for GC?

Answer:

EXAMPLE / FLOW

```
No.
```

The static reference still keeps it reachable.

✦ Practical Exercise — Memory Retention

```C#
public static readonly List<Gift>
    Gifts = new();
```

Loop:

```C#
while (true)
{
    Gifts.Add(
        new Gift());
}
```

Can GC simply remove old gifts?

Answer:

```C#
No,
because the static list
still references them.
```

✦ Practical Exercise — Race Condition

```C#
public static int Count;
```

Two threads execute:

```C#
Count++;
```

Is this automatically thread-safe?

Answer:

EXAMPLE / FLOW

No.

`static` does not provide atomicity or synchronization.

✦ Interview Questions

What is const in C#?

`const` declares a compile-time constant.

The value must be assigned in the declaration, must be computable as a constant expression, and cannot later change.

Does const belong to the object or type?

A constant field belongs to the type rather than each instance.

It behaves like a static member even though the `static` keyword is not written.

Can we write static const?

No.

```C#
static const int Value = 1;
```

is invalid.

A constant field is already type-level.

Can const be initialized in a constructor?

No.

A `const` must be initialized at the declaration.

Can const use runtime values?

No.

This is invalid:

```C#
const DateTime Started =
    DateTime.UtcNow;
```

because the value is not known at compile time.

What is readonly?

`readonly` is a field modifier that restricts direct assignment to initialization time.

An instance readonly field can be assigned at declaration or in an instance constructor of the containing type.

What is the main difference between const and readonly?

`const`:

EXAMPLE / FLOW

```
Compile-time value
```

`readonly`:

EXAMPLE / FLOW

```
Runtime initialization allowed
```

A readonly field can also have different values for different object instances.

Can readonly be assigned in a constructor?

Yes.

For an instance readonly field:

EXAMPLE / FLOW

```
field declaration
or
instance constructor
```

are the main allowed assignment locations.

Does readonly make a reference object immutable?

No.

Example:

C#

```
readonly List<int> Values;
```

means the `Values` field cannot later point to another list.

The existing list can still be modified unless its type/design prevents mutation.

What is static?

`static` means a member belongs to the type rather than a specific instance.

One shared static field exists for the relevant type instead of one per object.

What is the difference between static and instance fields?

Instance field:

EXAMPLE / FLOW

```
one per object
```

Static field:

EXAMPLE / FLOW

```
one shared by type
```

What is static readonly?

`static readonly` defines a shared type-level field that can be initialized at runtime but cannot later be directly reassigned outside its permitted initialization context.

Example:

C#

```
public static readonly DateTime
    StartedAt =
        DateTime.UtcNow;
```

What is the difference between const and static readonly?

`const`:

EXAMPLE / FLOW

```
compile-time constant
```

`static readonly`:

EXAMPLE / FLOW

```
runtime-initialized shared field
```

Additionally, public const values can be embedded in separately compiled callers, while static readonly values are read at runtime.

Why can changing a public const be dangerous?

Consumers can compile the constant value into their own assembly.

Changing the library's const value without recompiling consumers can leave old values in those consumers.

What is a static class?

A static class:

C#

```
cannot be instantiated

contains only static members

is implicitly sealed
```

and is accessed through its type name.

Can a static class have an instance field?

No.

Its members must be static.

Can a static class be inherited?

No.

It is implicitly sealed.

Can static methods be overridden?

No, normal static methods belong to the type rather than object instances and do not participate in runtime instance polymorphic overriding. Microsoft specifically notes static methods can be overloaded but not overridden in the ordinary instance-polymorphism sense.

What is a static constructor?

A static constructor initializes type-level state.

The CLR calls it automatically before the type's first relevant use, and it runs at most once for that type initialization context.

Can a static constructor have parameters?

No.

Can a static constructor have an access modifier?

No.

C# controls its accessibility and runtime invocation.

Can we call a static constructor manually?

No.

The runtime invokes it automatically.

What happens if a static constructor throws?

The type initialization fails, commonly surfacing as `TypeInitializationException`, and the runtime does not simply rerun that constructor on every subsequent access.

Should static constructors perform network/database calls?

Normally no.

Heavy blocking or failure-prone operations make type initialization fragile and can cause initialization failures or deadlock risk.

Is static automatically thread-safe?

No.

Static means shared.

Shared mutable state may actually increase concurrency risks.

Is static readonly automatically thread-safe?

No.

The field cannot be reassigned, but the referenced object may remain mutable.

What is object lifetime?

An object's lifetime begins when it is created and continues while it remains reachable by the running application.

When it is no longer reachable from GC roots, it becomes eligible for garbage collection.

What are GC roots?

GC roots are references from places the garbage collector treats as starting points for reachability.

Examples include:

EXAMPLE / FLOW

```
Static fields

Active stack references

GC handles

Finalization-related roots
```

Can a static field prevent an object from being collected?

Yes.

If a static field strongly references an object, that object remains reachable through the static root.

Does setting one reference to null destroy the object?

No.

If other references still point to the object, it remains alive.

Does scope equal lifetime?

No.

Scope describes where a variable name can be referenced in source code.

Lifetime describes how long the runtime object remains reachable/alive.

Does .NET garbage collection mean memory leaks are impossible?

No.

If objects remain reachable unnecessarily—for example through a growing static collection—the GC correctly considers them live and cannot reclaim them.

What is the difference between static and singleton?

Static:

EXAMPLE / FLOW

```
type-level member/state
```

Singleton:

EXAMPLE / FLOW

```
one object instance
```

A singleton can implement interfaces, participate in dependency injection, receive dependencies, and have instance semantics.

They are not the same concept.

✦ Best Practices

Use `const` only for values that are truly compile-time constants.

Use `readonly` for instance state that should be fixed after object construction.

Use `static readonly` for shared values that require runtime initialization.

Prefer constructor injection plus private readonly fields for service dependencies.

Avoid public mutable fields.

Avoid mutable static global state.

Do not use static storage for:

EXAMPLE / FLOW

```
Current user

Current tenant

Request data

DbContext

Per-request state
```

in ASP.NET Core applications.

Do not assume:

EXAMPLE / FLOW

```
readonly = immutable
```

or:

C#

```
static = thread-safe
```

or:

```C#
static readonly = immutable + thread-safe
```

because none of those statements is generally true.

Use configuration for values that differ by:

EXAMPLE / FLOW

```
environment

deployment

operations
```

rather than hardcoding them as constants.

Be cautious when exposing public `const` fields from reusable libraries because of compile-time value embedding.

Prefer simple inline static initialization over explicit static constructors when no complex initialization is required.

Avoid heavy work inside static constructors.

Remember that static references can extend object lifetime and cause memory retention.

Use dependency injection rather than static service locators/global service classes.

Use immutable or thread-safe data structures for shared application-wide state when appropriate.

✦ Quick Revision

Concept	Meaning
Field	Variable belonging to a type/object
Instance field	One copy per object
Static field	Shared by type
`const`	Compile-time constant
`readonly`	Reassignment restricted after initialization
`static readonly`	Shared runtime-initialized readonly field
Static method	Method belonging to type
Instance method	Method belonging to object
Static class	Non-instantiable type containing static members
Static constructor	One-time type initialization
Object lifetime	Time object remains reachable/live
GC root	Starting reference used by GC reachability analysis
Static root	Static field/data keeping object reachable
Scope	Where variable name can be used
Lifetime	How long object/value remains alive
Immutable	Object state cannot change
Thread-safe	Safe for intended concurrent use
Race condition	Concurrent operations produce incorrect result
DI singleton	One managed service object instance
Static state	Type-level shared state

✦ Complete Fundraising Example

C#

```
public static class GiftRules
{
    // Compile-time constant.
    public const decimal MinimumGiftAmount =
        0.01m;

    // Runtime-created shared value.
    public static readonly TimeSpan
        DefaultProcessingTimeout =
            TimeSpan.FromSeconds(
                30);
}

public interface IGiftRepository
{
    void Save(
        Gift gift);
}

public sealed class GiftService
{
    private readonly IGiftRepository
        _giftRepository;

    private readonly long
        _tenantId;

    public GiftService(
        IGiftRepository giftRepository,
        long tenantId)
    {
        _giftRepository =
            giftRepository;

        _tenantId =
            tenantId;
    }

    public void CreateGift(
        decimal amount)
    {
        if (amount <
            GiftRules.MinimumGiftAmount)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        var gift =
            new Gift
            {
                TenantId =
                    _tenantId,

                TotalAmount =
                    amount
            };

        _giftRepository.Save(
            gift);
    }
```

```
}

public sealed class Gift
{
    public long TenantId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        init;
    }
}
```

Understanding:

```
C#
MinimumGiftAmount
    →
const
    →
compile-time rule

DefaultProcessingTimeout
    →
static readonly
    →
runtime-created
    →
one shared value

_giftRepository
    →
readonly
    →
assigned during GiftService construction
    →
same repository reference for service lifetime

_tenantId
    →
readonly
    →
per GiftService instance

Gift.TotalAmount
    →
instance state
    →
belongs to each Gift object
```

✦ Complete Concept Flow

EXAMPLE / FLOW

```
Need a Value
    →
Ask:
Who owns it?
```

If each object needs its own value:

EXAMPLE / FLOW

```
Object Instance
    →
Instance Field
```

If all objects share one value:

C#

```
Type
    →
static
```

Then ask:

EXAMPLE / FLOW

```
Can the value change?
```

If it must be a true compile-time constant:

EXAMPLE / FLOW

```
Known at compile time
    →
const
```

If created at runtime but fixed after object construction:

EXAMPLE / FLOW

```
Per object
    →
readonly
```

If created at runtime, shared by type, and fixed after type initialization:

C#

```
Shared
    →
static readonly
```

✦ Lifetime Flow

EXAMPLE / FLOW

```
new Gift()
    →
Gift object created
    →
References point to it
    →
Object is reachable
```

Later:

EXAMPLE / FLOW

```
References removed
    →
Any remaining GC-root path?
```

If yes:

EXAMPLE / FLOW

```
Object remains alive
```

If no:

EXAMPLE / FLOW

```
Object becomes eligible for GC
```

Static reference example:

C#

```
static field
    →
Gift object
    →
still reachable
    →
GC keeps object
```

Remove static reference:

C#

```
static field = null
    →
No other roots?
    →
Object becomes eligible for collection
```

✦ ASP.NET Core Design Flow

C#
```
Need pure calculation?
    →
static method may fit
```

EXAMPLE / FLOW
```
Need compile-time invariant?
    →
const may fit
```

C#
```
Need per-service injected dependency?
    →
private readonly field
```

C#
```
Need application-wide service?
    →
DI singleton
rather than automatically using static
```

C#
```
Need request-specific state?
    →
scoped service
NOT mutable static field
```

The most important lesson from this topic is:

☀ **`const` describes compile-time fixed values, `readonly` restricts reassignment after initialization, and `static` controls ownership and sharing at the type level. These are separate concepts, and combining them correctly requires thinking about mutability, initialization time, sharing, threading, versioning, and object lifetime.**

A professional C# developer should be able to answer not only:

C#
```
"What does const/readonly/static mean?"
```

but also:

EXAMPLE / FLOW

```
"When should I use each?"

"Could this state be shared accidentally?"

"Will this keep objects alive?"

"Is it thread-safe?"

"Should this really be configuration?"

"Should DI manage this instead?"
```

Next roadmap topic: Strings and String Processing.